PRS Pricing Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 26-March-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

1 Test Results — PRS Pricing Model (MKM-PR-001)

Tests run on **26 March 2026 at 18:58:34**.

Table 1: Test Results Summary — PRS Pricing Model (MKM-PR-001)

Total Tests	61
Passed	61
Failed	0
Skipped	0
Duration	4.94s

Table 2: Detailed Test Results — PRS Pricing Model

Test Description	Acceptance Criteria	Result	Time
Basis waterfall in generated output	`bw1["model_uncertainty_bp"] == 2.0`; `bw1["terrain_bp"] == 2.0` (+6 more)	Pass	0.447s
Composition flat penalty	`result["composition_bp"] == COMPOSITION_BASIS_BPS["Flat"]`	Pass	0.000s
Composition pre2000 penalty	`result["composition_bp"] == 1.0`	Pass	0.000s
Distance basis capped	`result["distance_bp"] == DISTANCE_MAX_BPS`	Pass	0.000s
Distance basis scales linearly	`result["distance_bp"] == pytest.approx(1.0, abs=0.1)`	Pass	0.000s
Elevation above gauge reduces spread	`result["elevation_bp"] == pytest.approx(-1.0, abs=0.1)`	Pass	0.000s
Elevation below gauge no benefit	`result["elevation_bp"] == 0.0`	Pass	0.000s
Elevation capped at 3bp	`result["elevation_bp"] == -ELEVATION_MAX_BENEFIT_BPS`	Pass	0.000s
Model uncertainty is fixed	`result["model_uncertainty_bp"] == MODEL_UNCERTAINTY_BPS`	Pass	0.000s
Terrain zone1 low risk	`result["terrain_bp"] == 0.0`	Pass	0.000s
Terrain zone3 high risk	`result["terrain_bp"] == TERRAIN_BASIS_BPS["Zone 3a"]`	Pass	0.000s
Total basis sums components	`result["total_basis_bp"] == pytest.approx(expected_total,...`	Pass	0.000s
1Y from Feb 2026 = last Friday May 2027.	`mat == last_friday_of_month(2027, 5)`	Pass	0.000s
3Y from Feb 2026 = last Friday May 2029.	`mat == last_friday_of_month(2029, 5)`; `mat.month == 5` (+1 more)	Pass	0.000s
Jan 2027 uses Aug 2026 roll → Nov 2029 maturity.	`mat == last_friday_of_month(2029, 11)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
3Y from Sep 2026 = last Friday Nov 2029 (Aug roll).	`mat == last_friday_of_month(2029, 11); mat.month == 11`	Pass	0.000s
5Y from Feb 2026 = last Friday May 2031.	`mat == last_friday_of_month(2031, 5)`	Pass	0.000s
5Y from Sep 2026 = last Friday Nov 2031.	`mat == last_friday_of_month(2031, 11)`	Pass	0.000s
All maturities should fall on a Friday.	`mat.weekday() == 4`	Pass	0.000s
3Y maturity jumps from May to Nov at the Aug roll.	`may_mat.month == 5; nov_mat.month == 11` (+1 more)	Pass	0.000s
Aug onwards uses Aug roll.	`roll == last_friday_of_month(2026, 8); roll.month == 8`	Pass	0.000s
December uses the same year's Aug roll.	`roll.month == 8; roll.year == 2026`	Pass	0.000s
Feb-Jul uses Feb roll.	`roll == last_friday_of_month(2026, 2); roll.month == 2`	Pass	0.000s
January uses the previous year's Aug roll.	`roll == last_friday_of_month(2026, 8); roll.month == 8` (+1 more)	Pass	0.000s
July is the last month of the Feb roll period.	`roll.month == 2; roll.year == 2026`	Pass	0.000s
March is still in the Feb roll period.	`roll.month == 2; roll.year == 2026`	Pass	0.000s
compute_maturity_date with no ref_date uses today.	`isinstance(mat, date); mat.weekday() == 4`	Pass	0.000s
current_roll_date(None) uses today and returns a valid date.	`isinstance(roll, date); roll.weekday() == 4`	Pass	0.000s
maturity_table with no ref_date uses today.	`len(table) == 5; all(isinstance(e['maturity_date'], date) for e in table)`	Pass	0.000s
Empty propertyts dir	`stats["total_properties"] == 0; stats["properties_with_gev"] == 0`	Pass	0.001s
Generator should still work without gauge hazard curves (empty basis).	`stats["properties_with_gev"] == 2; stats["properties_with_floor"] == 1` (+1 more)	Pass	0.425s
No propertyts dir	—	Pass	0.000s
Aug 2026: 31st is Monday, last Friday = 28th.	`last_friday_of_month(2026, 8) == date(2026, 8, 28)`	Pass	0.000s
Feb 2026: 28th is Saturday, last Friday = 27th.	`last_friday_of_month(2026, 2) == date(2026, 2, 27)`	Pass	0.000s
May 2029: 31st is Thursday, last Friday = 25th.	`last_friday_of_month(2029, 5) == date(2029, 5, 25)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Nov 2029: 30th is Friday.	`last_friday_of_month(2029, 11) == date(2029, 11, 30)`	Pass	0.000s
Result should always be a Friday (weekday=4).	`d.weekday() == 4`	Pass	0.000s
Actual years should be strictly increasing.	`years[i] > years[i - 1]`	Pass	0.000s
All maturities should be in May for the Feb roll.	`e['maturity_date'].month == 5`	Pass	0.000s
All maturities should be in November for the Aug roll.	`e['maturity_date'].month == 11`	Pass	0.000s
Maturity string should be formatted as 'DD Mon YYYY'.	`len(e['maturity_str']) > 8; 'May' in e['maturity_str'] or 'Nov' in e['maturity_str']`	Pass	0.000s
Default table has 5 entries (1Y through 5Y).	`len(table) == 5`	Pass	0.000s
Tenors should be 1, 2, 3, 4, 5.	`tenors == [1, 2, 3, 4, 5]`	Pass	0.000s
Positive spread	`spread > 0; 50 < spread < 500`	Pass	0.000s
Spread increases with hazard	`spread_high > spread_low`	Pass	0.000s
Spread values at different tenors	`s > 0`	Pass	0.000s
Zero hazard rate	`spread == 0.0`	Pass	0.000s
Basis calculation	`len(nearest) > 0; ng0["gauge_id"] == "GAUGE-001"` (+5 more)	Pass	0.404s
Depth thresholds	`"any_flood" in thresholds; "moderate" in thresholds` (+4 more)	Pass	0.404s
Generate produces output	`output_path.exists(); "metadata" in data` (+2 more)	Pass	0.413s
Gev params structure	`"shape" in gev; "loc" in gev` (+2 more)	Pass	0.405s
High transmission rate	`prop3["summary"]["flood_transmission_rate"] == 1.0`	Pass	0.405s
Output structure	`"PROP-001" in curves; "PROP-002" in curves` (+21 more)	Pass	0.404s
Properties with gev	`stats["properties_with_gev"] == 2; stats["properties_with_floor"] == 1` (+1 more)	Pass	0.409s
Survival decreases with tenor	`survival[i] <= survival[i - 1]`	Pass	0.404s
Term structure	`ts["tenors"] == TENORS; threshold_name in ts` (+3 more)	Pass	0.403s
Recovery rates by trigger	`len(spreads) > 0`	Pass	0.409s
Recovery reduces spread	`spread_with_recovery < spread_no_recovery`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
All tenors should round-trip through maturity and back.	`tenor_from_maturity(mat, ref) == t`	Pass	0.000s
A date not in the table returns the closest tenor.	`result in [1, 2, 3, 4, 5]`	Pass	0.000s
Exact maturity date returns correct tenor.	`tenor_from_maturity(mat, date(2026, 2, 24)) == 3`	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (30 tests)	D. Kelly
07-Mar-2026	1.1	57 test(s) added; 30 test(s) removed (total: 57)	D. Kelly
07-Mar-2026	1.2	30 test(s) added (total: 87)	D. Kelly
18-Mar-2026	1.3	4 test(s) added (total: 91)	D. Kelly
26-Mar-2026	1.4	30 test(s) removed (total: 61)	D. Kelly